WIE

Notification Service

Complete API & Real-Time Socket Documentation for React Native Developers

REST API • Socket.IO Events • Notification Types • React Native Setup

Base URL: http://localhost:5006/api/notification • Socket: http://localhost:5006 • v1.0

Table of Contents

01 Service Overview & Base URL

The WIE Notification Service runs on port 5006. It delivers real-time push notifications to users via Socket.IO and persists them in MongoDB so users can review them later. Notifications are triggered by other WIE microservices (Chat, Follow, Media, Transaction) via RabbitMQ or direct HTTP. The React Native app receives them instantly over a WebSocket connection.

Service Config

```
Base URL (development) : http://localhost:5006/api/notification
Base URL (Android emu) : http://10.0.2.2:5006/api/notification
Socket.IO URL : http://10.0.2.2:5006
All REST routes require Bearer token (authenticateToken middleware).
Socket.IO connection also requires Bearer token via auth handshake.
```

02 Authentication

All REST endpoints and the Socket.IO connection require a valid JWT Bearer token obtained at login from the User Service (port 5005). In React Native, store the token in AsyncStorage.

TypeScript

```typescript
// src/services/notificationService.ts
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';
import Config from 'react-native-config';
const BASE = Config.NOTIFICATION_API_URL || 'http://10.0.2.2:5006/api/notification';
export const notifApi = axios.create({ baseURL: BASE, timeout: 15000 });
notifApi.interceptors.request.use(async cfg => {
const t = await AsyncStorage.getItem('token');
if (t) cfg.headers.Authorization = `Bearer ${t}`;
return cfg;
});
```

A 401 response means the token is expired or missing. Clear AsyncStorage and redirect to the Login screen.

03 Notification Data Model

Every notification object returned by the API follows this shape. Understanding it helps you correctly render notification items and badge counts in your UI.

Field	Type	Req.	Description
`_id`	`string`	—	Unique notification ID (MongoDB ObjectId).
`userId`	`string`	—	Recipient user ID.
`type`	`string`	—	Notification type (see Section 04 for all values).
`title`	`string`	—	Short notification headline.
`message`	`string`	—	Full notification body text.
`isRead`	`boolean`	—	False = unread. True = user has seen it.

Field	Type	Req.	Description
fromUserId	string?	—	Sender user ID (for follow, like, comment, etc).
ticketId	string?	—	Related event ticket ObjectId.
groupId	string?	—	Related event group ObjectId.
bookingId	string?	—	Related booking ID string.
chatId	string?	—	Related chat ID.
eventName	string?	—	Event name for display without extra lookup.
link	string?	—	Deep-link path (e.g. /post/abc, /bookings/xyz).
metadata	Map?	—	Flexible key-value map of extra data (avatars, counts, URLs).
isGrouped	boolean	—	True if multiple actors are grouped into one notification.
primaryActors	Actor[]	—	Enriched actor list for grouped notifications.
othersCount	number	—	How many additional actors beyond primaryActors.
lastUpdatedAt	string	—	ISO 8601 — when the grouped notification was last updated.
createdAt	string	—	ISO 8601 creation timestamp.

metadata Map — Common Keys

Key	Description
postId	For post-related notifications (like, comment, share).
fluxId	For flux/story notifications (mention, react, screenshot).
fluxMediaUrl	Story media URL for preview thumbnail.
likerName	Display name of the person who liked.
likerAvatar	Profile picture URL of the actor.
commenterName	Display name of the commenter.
bookingId	Booking ID for ticket/payment notifications.
refundAmount	Refund amount in INR.
screenshotCount	Total screenshot count for flux_screenshot type.
callerName	Name of user who mentioned/tagged.
targetUrl	Deep link URL for navigating from notification.

04 Notification Types — Full Reference

The type field is a strict enum. The following table lists every valid notification type, which WIE service generates it, and what triggers it.

Social — Follow

type value	Triggered by / When
`following`	[Follow Service] User A started following User B (public account).
`follow_request`	[Follow Service] User A sent a follow request to private User B.
`follow_accepted`	[Follow Service] A pending follow request was accepted.

Social — Posts

type value	Triggered by / When
`post_liked`	[Media Service] Someone liked the user's post.
`post_reacted`	[Media Service] Someone reacted with a non-❤■ emoji to the post.
`post_commented`	[Media Service] Someone added a comment to the user's post.
`post_replied`	[Media Service] Someone replied to the user's comment on a post.
`post_shared`	[Media Service] Someone shared the user's post via DM.
`mention`	[Media Service] Someone tagged or @mentioned the user in a post.
`like`	[Media Service] Generic like (non-post-specific).
`comment`	[Media Service] Generic comment notification.

Social — Flux / Stories

type value	Triggered by / When
`flux_mention`	[Media Service] Flux owner mentioned this user in their story.
`flux_mention_sent`	[Media Service] Confirmation: you mentioned someone in your story.
`flux_remention`	[Media Service] A mentioned user reshared the owner's story.
`flux_remention_sent`	[Media Service] Confirmation: you reshared a story you were mentioned in.
`flux_comment`	[Media Service] Someone commented on the user's flux.
`flux_like`	[Media Service] Someone liked the user's flux.
`flux_reply`	[Media Service] Someone replied to the user's flux via DM.
`flux_share`	[Media Service] Someone shared the user's flux via DM.
`flux_screenshot`	[Media Service] Someone took a screenshot of the user's flux.

Messaging

type value	Triggered by / When
`message_received`	[Chat Service] New chat message received (used for push badge).

Events — Booking & Payment

type value	Triggered by / When
booking_confirmed	[Transaction Svc] Booking confirmed (free or paid event registered).
payment_success	[Transaction Svc] Razorpay payment verified and booking confirmed.
payment_failed	[Transaction Svc] Payment attempt failed.
payment_processing	[Transaction Svc] Payment is being processed.
booking_cancelled	[Transaction Svc] Booking cancelled by user or admin.
ticket_purchased	[Transaction Svc] Ticket purchase completed.
ticket_cancelled	[Transaction Svc] Ticket cancelled.
ticket_verified	[Transaction Svc] Ticket QR scanned and verified at event.
qr_code_generated	[Transaction Svc] QR code generated for a booking.

Events — Refunds

type value	Triggered by / When
refund_initiated	[Transaction Svc] Refund initiated after cancellation.
refund_processing	[Transaction Svc] Refund is being processed by Razorpay.
refund_completed	[Transaction Svc] Refund credited to user's account.
refund_failed	[Transaction Svc] Refund failed — will be processed manually.
event_refunded	[Notification Svc] Event was cancelled and refund issued by host.

Events — Host Actions

type value	Triggered by / When
event_created	[Ticket Service] A new event was created by a group the user follows.
event_hosted	[Ticket Service] An event the user has a booking for has started.
event_cancelled	[Ticket Service] An event was cancelled by the host.
event_completed	[Ticket Service] An event has ended.
event_updated	[Ticket Service] Event details (date/venue) were updated by host.
event_rehosted	[Ticket Service] Cancelled event has been rescheduled.
event_recovered	[Ticket Service] A previously cancelled event was restored.
event_reminder	[Ticket Service] Reminder: event is starting soon.
event_invite	[Ticket Service] User was invited to an event by the organiser.
group_updated	[Ticket Service] An organiser group the user follows was updated.

System

type value	Triggered by / When
system	[Any service] Generic system notification (announcements, maintenance).

05 Get Notifications

Returns a paginated list of notifications for the authenticated user, sorted newest-first. Includes unread count and category counts for badge rendering.

Method	Endpoint	Auth	Description
GET	/get-notifications	■	Fetch notifications. Optional type filter and pagination.

Query Parameters

Field	Type	Req.	Description
type	string	No	Filter by notification type. Pass 'all' or omit to get all types.
limit	number	No	Max notifications to return. Default: 50.
skip	number	No	Number of notifications to skip (offset). Default: 0.

Response (HTTP 200)

JSON Response


```
{
"notifications": [
{
"_id": "notif_abc123",
"userId": "user_me",
"type": "post_liked",
"title": "Jane Doe liked your post",
"message": "Jane Doe liked your post ❤■",
"isRead": false,
"fromUserId": "user_jane",
"link": "/post/post_xyz",
"metadata": {
"postId": "post_xyz",
"likerName": "Jane Doe",
"likerAvatar": "https://cdn.../avatar.jpg",
"emoji": "❤■"
},
"createdAt": "2025-12-01T11:00:00.000Z"
}
],
"unreadCount": 7,
"counts": {
"all": 50,
"events": 3,
"invites": 1
}
}
```


Use the unreadCount field to update the badge on the Notifications tab icon. Call this endpoint on app focus and whenever you receive a new-notification socket event.

06 Mark Single Notification as Read

Marks one specific notification as read and emits a real-time socket event back to the user with the updated unread count.

Method	Endpoint	Auth	Description
PATCH	/notification-read/:notificationId	■	Mark one notification as read. Emits notification-read socket event.

URL Parameter

Parameter	Description
:notificationId	MongoDB ObjectId of the notification to mark as read.

Response (HTTP 200)

JSON Response

```
{
"message": "Notification marked as read",
"notification": { /* updated notification object with isRead: true */ }
}
```

Error Responses

Status	Meaning
400 Bad Request	notificationId is missing or not a valid MongoDB ObjectId.
404 Not Found	Notification does not exist or belongs to a different user.

07 Mark All Notifications as Read

Marks every unread notification for the authenticated user as read in a single operation. Emits a socket event with unreadCount: 0 so the badge clears instantly.

Method	Endpoint	Auth	Description
PATCH	/mark-all-read	■	Mark ALL unread notifications as read. Badge count resets to 0.

Response (HTTP 200)

JSON Response

```
{ "message": "All notifications marked as read" }
```

Call this when the user taps 'Mark all as read' in the notification list header. The all-notifications-read socket event is also emitted so other open tabs/devices update instantly.

08 Delete Notification

Permanently deletes a single notification. Emits a socket event with the updated unread count. Deletion is irreversible.

Method	Endpoint	Auth	Description
DELETE	/delete-notification/:notificationId	■	Permanently delete a notification. Emits notification-deleted socket event.

Response (HTTP 200)

JSON Response

```
{ "message": "Notification deleted successfully" }
```

09 Create Notification (Internal / Server-to-Server)

This endpoint is used internally by other WIE microservices to create notifications. You typically do NOT call this from the mobile app — notifications are created server-side by Chat, Follow, Media, and Transaction services. It is documented here for completeness and debugging.

Method	Endpoint	Auth	Description
POST	/create-notification	■	Create a notification. Used by backend services, not the mobile client.

Request Body — application/json

Field	Type	Req.	Description
userId	string	Yes	Recipient user ID.
type	string	Yes	One of the valid type enum values (see Section 04).
title	string	Yes	Short headline. E.g. 'Jane liked your post'.
message	string	Yes	Full body text.
fromUserId	string	No	Actor user ID (sender, liker, follower, etc).
ticketId	string	No	Event ticket ObjectId.
groupId	string	No	Event group ObjectId.
bookingId	string	No	Booking ID string.
eventName	string	No	Event name for quick display.
link	string	No	Deep link path for navigation (e.g. /post/abc).
metadata	object	No	Key-value extra data (avatars, counts, URLs).

Response (HTTP 201)

JSON Response

```
{
"success": true,
"notification": { /* full notification object */ }
}
```

After saving the notification, the server automatically emits a new-notification socket event to the user's personal Socket.IO room. The React Native client receives it in real time without polling.

10 Real-Time Socket.IO Events

The notification service uses Socket.IO for real-time delivery. Connect with your JWT token and join your personal user room. You will then receive instant events whenever any notification action occurs.

Connecting to Socket.IO from React Native

TypeScript

```
// Install: npm install socket.io-client
import { io, Socket } from 'socket.io-client';
import AsyncStorage from '@react-native-async-storage/async-storage';
import Config from 'react-native-config';

let socket: Socket | null = null;

export const connectNotificationSocket = async () => {
const token = await AsyncStorage.getItem('token');
if (!token) return;

socket = io(Config.NOTIFICATION_SOCKET_URL || 'http://10.0.2.2:5006', {
transports: ['websocket'],
auth: { token }, // ← JWT sent here for authentication
reconnection: true,
reconnectionAttempts: 10,
reconnectionDelay: 2000,
});

socket.on('connect', () => {
console.log('■ Notification socket connected:', socket?.id);
});

socket.on('disconnect', reason => {
console.warn('■ Notification socket disconnected:', reason);
});

return socket;
};

export const disconnectNotificationSocket = () => {
socket?.disconnect();
socket = null;
};
```

Listening to Notification Events

TypeScript

```
// In your NotificationContext or useNotifications hook:
import { useEffect } from 'react';

const useNotificationSocket = () => {
const [unreadCount, setUnreadCount] = useState(0);
const [notifications, setNotifications] = useState([]);

useEffect(() => {
const sock = getSocket(); // your singleton socket instance

// ■■ New notification arrives ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sock.on('new-notification', ({ notification, unreadCount }) => {
setNotifications(prev => [notification, ...prev]);
setUnreadCount(unreadCount);
// Show in-app toast/banner
showToast(notification.title, notification.message);
```

```
});
// ■■ Single notification marked read ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sock.on('notification-read', ({ notificationId, unreadCount }) => {
setNotifications(prev =>
prev.map(n => n._id === notificationId ? { ...n, isRead: true } : n)
);
setUnreadCount(unreadCount);
});
// ■■ All notifications marked read ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sock.on('all-notifications-read', ({ unreadCount }) => {
setNotifications(prev => prev.map(n => ({ ...n, isRead: true })));
setUnreadCount(unreadCount); // always 0
});
// ■■ Notification deleted ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sock.on('notification-deleted', ({ notificationId, unreadCount }) => {
setNotifications(prev => prev.filter(n => n._id !== notificationId));
setUnreadCount(unreadCount);
});
return () => {
sock.off('new-notification');
sock.off('notification-read');
sock.off('all-notifications-read');
sock.off('notification-deleted');
};
}, []);
return { unreadCount, notifications };
};
```

11 Socket Events — Full Reference Table

Inbound events — emitted BY the server TO the client.

Event Name	Description & Payload
`new-notification`	A new notification was created. Payload: { notification, unreadCount }
`notification-read`	One notification was marked as read. Payload: { notificationId, unreadCount }
`all-notifications-read`	All notifications marked read. Payload: { unreadCount: 0 }
`notification-deleted`	A notification was deleted. Payload: { notificationId, unreadCount }

new-notification Payload — Full Shape

JSON Payload

```
{
"notification": {
"_id": "notif_abc",
"userId": "user_me",
"type": "post_liked",
"title": "Jane Doe liked your post",
"message": "Jane Doe liked your post ❤■",
"isRead": false,
"fromUserId": "user_jane",
"link": "/post/post_xyz",
"metadata": {
"postId": "post_xyz",
"likerName": "Jane Doe",
"likerAvatar": "https://cdn.../avatar.jpg"
},
"createdAt": "2025-12-01T11:00:00.000Z"
},
"unreadCount": 8
}
```

How to Handle Navigation from a Notification

TypeScript

```
// Use the 'link' field to determine where to navigate
const handleNotificationPress = (notification) => {
const link = notification.link || '';
// Mark as read first
notifApi.patch(`/notification-read/${notification._id}`);
// Navigate based on link path
if (link.startsWith('/post/')) {
navigation.navigate('PostDetail', { postId: link.split('/post/')[1] });
} else if (link.startsWith('/bookings/')) {
navigation.navigate('BookingDetail', { bookingId: link.split('/bookings/')[1] });
} else if (link.startsWith('/profile/')) {
navigation.navigate('Profile', { username: link.split('/profile/')[1] });
} else if (link.startsWith('/post/flux-view')) {
const params = new URLSearchParams(link.split('?')[1]);
navigation.navigate('FluxViewer', { fluxId: params.get('fluxId') });
```

```
} else {
navigation.navigate('Notifications');
}
};
```

12 React Native Integration Examples

Complete notificationService.ts

TypeScript

```
// src/services/notificationService.ts
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';
import { io, Socket } from 'socket.io-client';
import Config from 'react-native-config';
const BASE = Config.NOTIFICATION_API_URL || 'http://10.0.2.2:5006/api/notification';
const SOCKET = Config.NOTIFICATION_API_URL?.replace('/api/notification','') || 'http://10.0.2.2:5006';
export const notifApi = axios.create({ baseURL: BASE, timeout: 15000 });
notifApi.interceptors.request.use(async cfg => {
const t = await AsyncStorage.getItem('token');
if (t) cfg.headers.Authorization = `Bearer ${t}`;
return cfg;
});
// ■■ REST Helpers ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const getNotifications = (type?: string, limit = 50, skip = 0) =>
notifApi.get('/get-notifications', { params: { type, limit, skip } }).then(r => r.data);
export const markOneRead = (id: string) =>
notifApi.patch(`/notification-read/${id}`).then(r => r.data);
export const markAllRead = () =>
notifApi.patch('/mark-all-read').then(r => r.data);
export const deleteNotification = (id: string) =>
notifApi.delete(`/delete-notification/${id}`).then(r => r.data);
// ■■ Socket ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
let socket: Socket | null = null;
export const connectSocket = async () => {
if (socket?.connected) return socket;
const token = await AsyncStorage.getItem('token');
socket = io(SOCKET, {
transports: ['websocket'],
auth: { token },
reconnection: true,
reconnectionAttempts: 10,
reconnectionDelay: 2000,
});
return socket;
};
export const getSocket = () => socket;
export const disconnectSocket = () => { socket?.disconnect(); socket = null; };
```

App.tsx — Connect on Login, Disconnect on Logout

TypeScript

```
// In your auth flow — connect socket after login:
const onLoginSuccess = async () => {
await connectSocket();
// ... navigate to home
};
```

```
// On logout:
const onLogout = () => {
disconnectSocket();
// ... clear token, navigate to Login
};
```

Notification Bell Icon with Badge

TypeScript

```
// src/components/NotificationBell.tsx
import { useEffect, useState } from 'react';
import { TouchableOpacity, View, Text } from 'react-native';
import { getNotifications, getSocket } from '../services/notificationService';
import Ionicons from '@expo/vector-icons/Ionicons';

export const NotificationBell = ({ onPress }) => {
const [unread, setUnread] = useState(0);

useEffect(() => {
// Load initial count
getNotifications('all', 1).then(data => setUnread(data.unreadCount));

const sock = getSocket();
if (!sock) return;

// Real-time updates
sock.on('new-notification', ({ unreadCount }) => setUnread(unreadCount));
sock.on('notification-read', ({ unreadCount }) => setUnread(unreadCount));
sock.on('all-notifications-read',({ unreadCount }) => setUnread(unreadCount));
sock.on('notification-deleted', ({ unreadCount }) => setUnread(unreadCount));

return () => {
sock.off('new-notification');
sock.off('notification-read');
sock.off('all-notifications-read');
sock.off('notification-deleted');
};
}, []);

return (
<TouchableOpacity onPress={onPress} style={{ padding: 8 }}>
<Ionicons name='notifications-outline' size={24} color='#fff' />
{unread > 0 && (
<View style={{ position:'absolute', top:4, right:4,
backgroundColor:'#DC2626', borderRadius:10, minWidth:18,
alignItems:'center', justifyContent:'center', paddingHorizontal:3 }}>
<Text style={{ color:'#fff', fontSize:10, fontWeight:'bold' }}>
{unread > 99 ? '99+' : unread}
</Text>
</View>
)}
</TouchableOpacity>
);
};
```

Notifications Screen — List with Infinite Scroll

TypeScript

```
// src/screens/NotificationsScreen.tsx
const LIMIT = 30;
```

```
const NotificationsScreen = () => {
const [items, setItems] = useState([]);
const [skip, setSkip] = useState(0);
const [hasMore, setHasMore] = useState(true);

const load = async (reset = false) => {
const currentSkip = reset ? 0 : skip;
const data = await getNotifications('all', LIMIT, currentSkip);
if (reset) setItems(data.notifications);
else setItems(prev => [...prev, ...data.notifications]);
setSkip(currentSkip + LIMIT);
setHasMore(data.notifications.length === LIMIT);
};

useEffect(() => { load(true); }, []);

return (
<FlatList
data={items}
keyExtractor={n => n._id}
renderItem={({ item }) => <NotificationItem n={item} />}
onEndReached={() => hasMore && load()}
onEndReachedThreshold={0.3}
ListHeaderComponent={
<TouchableOpacity onPress={markAllRead}>
<Text>Mark all as read</Text>
</TouchableOpacity>
}
/>
);
};
```

13 Full Route Reference

All routes are relative to http://localhost:5006/api/notification. All require Bearer token.

Method	Endpoint	Auth	Description
POST	/create-notification	■	Create a notification (server-to-server only). Emits socket event.
GET	/get-notifications	■	Get notifications. Optional ?type, ?limit, ?skip. Returns unreadCount.
PATCH	/notification-read/:notificationId	■	Mark one notification as read. Emits notification-read socket event.
PATCH	/mark-all-read	■	Mark all as read. Emits all-notifications-read socket event.
DELETE	/delete-notification/:notificationId	■	Delete a notification. Emits notification-deleted socket event.

Environment Variables (.env)

.env

```
# Notification Service URL
NOTIFICATION_API_URL=http://localhost:5006/api/notification
# For Android emulator:
# NOTIFICATION_API_URL=http://10.0.2.2:5006/api/notification
# Socket.IO connects to the base URL (without /api/notification):
# NOTIFICATION_SOCKET_URL=http://10.0.2.2:5006
```

Notification Type Quick Lookup

Notification Type	Navigation Action
post_liked / post_reacted	Navigate to the post using link: /post/:postId
post_commented / post_replied	Navigate to the post comments using link: /post/:postId
flux_mention / flux_like	Navigate to flux viewer using metadata.targetUrl
flux_screenshot	Navigate to your own flux using metadata.targetUrl
following / follow_request	Navigate to the sender's profile using link: /profile/:username
booking_confirmed	Navigate to booking detail using link: /bookings/:bookingId
payment_success / refund_*	Navigate to booking detail using link or metadata.bookingId
event_cancelled / rehosted	Navigate to event detail using metadata.ticketId
system	Show in-app alert or navigate to Notifications screen

The notification service uses RabbitMQ consumers for event-driven notifications (booking cancellations, refunds, event rehosting). These are created server-side automatically — the mobile client only needs the REST API and Socket.IO listeners described in this document.